

Breaking the Distribution

This report presents the practical engineering pathway by which AI systems can move beyond static training distributions toward adaptive social and interaction intelligence. It argues that pretraining, supervised fine-tuning, and offline preference optimization remain fundamentally open-loop processes: they imitate patterns already present in data but cannot reliably discover strategies outside that data. The central barrier is distributional invalidity, where learned feedback signals such as reward models degrade when policies explore outputs unlike those seen during training. Against this barrier, the report frames reinforcement learning post-training as a closed-loop control problem: a policy acts, an environment or verifier responds, and the policy updates from calibrated feedback.

The report traces this closed-loop pathway across two domains. In language models, it follows the progression from PPO and RLHF to GRPO, DAPO, and reinforcement learning with verifiable rewards, showing how the removal of fragile learned components — critics, reward models, and human annotation bottlenecks — enables models to explore beyond supervised demonstrations and discover new reasoning behaviors. In embodied systems, it examines SERL, HIL-SERL, and related safety extensions as the physical-world analogue: sample-efficient real-world reinforcement learning in which robots learn robust manipulation and recovery behaviors directly from interaction, guided by success classifiers, human corrections, and stability constraints.

By interpreting both LLM post-training and real-world robotic learning through control theory, the report identifies a shared architecture for out-of-distribution breakout: online exploration, calibrated feedback, and stability under policy drift. This architecture is presented as the practical bridge from physical intelligence to social intelligence, where agents must adapt in real time to novel partners, environments, and tasks. The report concludes that the decisive difference between systems that remain trapped inside their training distributions and systems that operate in the world is not scale alone, but the presence of a closed loop with a reliable sensor.

Post-Training, Sample-Efficient RL, and Control Theory as the Practical Pathway to Social Intelligence

Date: 2026-06-21 **Prepared for:** STAR Lab **Author:** Haobo Yang **Type:** Technical / Engineering Report
Position: Report 3 of 3 — the practical *how*

Related reports: - Report 1: *From Statistical Modeling to Emergent Understanding* (MBZUAI — the institutional *who*) - Report 2: *Compression, Prediction, Emergence* (the theoretical *why*)

Table of Contents

1. Prologue: The Distribution Barrier
2. Part I — The LLM Pathway: From Imitation to Exploration
 - 2.1 The Post-Training Stack (2026)
 - 2.2 PPO + RLHF: The Foundation
 - 2.3 GRPO: The Critic-Free Breakthrough
 - 2.4 DAPO: Stabilizing RL for Long Reasoning Trajectories
 - 2.5 RLVR: The Verifiable Reward Revolution
 - 2.6 The Emerging Frontier: Multi-Agent RL for LLMs
3. Interlude: The Control Theory Lens
4. Part II — The Embodied Pathway: SERL and Real-World RL
 - 4.1 The Real-World RL Problem
 - 4.2 SERL: The Breakthrough Framework
 - 4.3 HIL-SERL: Human-in-the-Loop

- 4.4 CIL-SERL and Safety Extensions
 - 4.5 The SERL-GRPO Parallel
5. Synthesis: Control Theory as the Practical Unification
 6. Coda: The Trilogy Complete
 7. References & Sources
-

1. Prologue: The Distribution Barrier

1.1 What Social Intelligence Demands

Eric Xing’s hierarchy of intelligence (Report 1, §2.4) identifies **Social Intelligence** as Level 2 — the threshold beyond physical understanding, where agents must:

- Distinguish **self from other** — recognize that other agents have different knowledge, goals, and beliefs
- **Collaborate** — divide labor, negotiate, coordinate toward shared goals
- **Adapt to novelty** — handle novel partners, novel environments, novel tasks that no training distribution could enumerate

This last requirement is the crux. Social intelligence is not about executing a known protocol with a known partner in a known setting. It is about **interacting successfully with agents, environments, and tasks you have never seen before**. And “never seen before” means, by definition, **out of distribution**.

1.2 The Pretraining Ceiling

Pretraining learns $P(token | context)$ from a static corpus — the entire internet, or some large subset of it. It is astonishingly powerful, but it has a hard ceiling: **it can only reproduce patterns that exist in its training data**. If the optimal strategy for a novel coordination problem is not latent in any text on the internet, pretraining cannot discover it.

Supervised Fine-Tuning (SFT) raises the ceiling slightly: it trains on expert demonstrations, so it can learn patterns that exist in demonstration data but not in the broader corpus. But SFT is still fundamentally imitative. The model learns to reproduce what experts did, not to **explore what experts did not do**.

Both pretraining and SFT are **open-loop** processes. The model produces outputs. The outputs are compared to a static target (next-token ground truth for pretraining, demonstration tokens for SFT). There is no feedback from an environment, no trial and error, no discovery through interaction. The model is bounded by the data it has seen.

1.3 The Distributional Invalidity Problem

The problem is not just that pretraining data is finite. The problem is that the **feedback signals themselves degrade** when the model operates outside its training envelope.

A 2025 evaluation study identified four failure modes of standard RLHF:

1. **Distributional Invalidity**: evaluation benchmarks (curated, clean, static) $\neq$ real deployment distributions (messy, shifting, adversarial)
2. **Temporal Invalidity**: reward models are evaluated *before* RL training, but they degrade on policy-generated OOD outputs *during* training
3. **Scope Invalidity**: single-turn evaluation cannot assess multi-step trajectory quality
4. **Process Invalidity**: output-only evaluation rewards correct-looking answers produced by flawed reasoning

The second failure mode — temporal invalidity — is the most fundamental. A reward model trained on human preference data is a **learned sensor**. Like any sensor, it has a calibration envelope. Inside that envelope, it provides reliable readings. Outside it — when the policy generates outputs unlike anything in

the preference data — the sensor miscalibrates. The feedback becomes noise. The policy, optimizing against noisy feedback, drifts further from the envelope. The sensor degrades further. This is a **runaway control failure**, and it is the central challenge of post-training.

1.4 The Thesis

Post-training via reinforcement learning is the mechanism by which models break out of their data distribution. Control theory is the framework that tells us when this breakout succeeds — and when it fails.

The argument proceeds in two parts, mirroring the two domains where the breakout is happening:

- **Part I — The LLM Pathway:** PPO → GRPO → RLVR. How post-training algorithms evolved to remove the learned sensor (reward model) and replace it with a **perfect sensor** (verifiable ground truth), enabling policy exploration beyond any human-labeled data.
- **Part II — The Embodied Pathway:** SERL and sample-efficient real-world RL. How the same control-theoretic pattern — closed-loop RL with verifiable feedback — enables robots to learn physical skills in minutes, directly in the real world, without simulation.

The synthesis shows that these are not two separate stories. They are **one architecture, applied to two domains** — and that architecture is control theory.

2. Part I — The LLM Pathway: From Imitation to Exploration

2.1 The Post-Training Stack (2026)

The modern post-training pipeline has consolidated into three stages:

Stage	Purpose	Methods	Exploration?
1. SFT	Teach instruction-following format and conversational style	1–10M curated examples	No — pure imitation
2. Preference Optimization	Align with human values, style, safety, refusals	DPO, SimPO, KTO, ORPO	No — offline preference pairs
3. RL with Verifiable Rewards	Push beyond training data; discover novel capabilities	GRPO, DAPO, RLVR, PPO	Yes — online trial and error

The first two stages are essential but bounded. **Only the third stage does exploration.** And the history of the third stage is a story of progressive simplification — what practitioners call the “deletion story”:

Step	What Was Removed	Algorithm
1	Second-order optimization math	PPO (from TRPO)
2	Learned Reward Model	DPO (replaced with implicit preference loss)
3	Reference Model	SimPO (average log-prob as implicit reward)
4	Separate SFT stage	ORPO (merged SFT + preference optimization)
5	Critic (Value Network)	GRPO (group-relative scoring)

Step	What Was Removed	Algorithm
6	Human annotators	RLVR (verifiable ground-truth rewards)

Each deletion removed a **component that could fail**. The direction of travel is toward minimalism: fewer moving parts, fewer learned models that can miscalibrate, fewer human judgments that can be gamed. The endgame is a system with only two components: a policy and a verifier. The policy explores. The verifier provides perfectly calibrated feedback. Nothing else can break.

2.2 PPO + RLHF: The Foundation

Proximal Policy Optimization (PPO) with **Reinforcement Learning from Human Feedback (RLHF)** established the template. The architecture keeps four models in memory simultaneously:

- **Policy model** (the LLM being trained — the *controller*)
- **Reference model** (frozen copy of the pretrained policy — the *setpoint*)
- **Reward model** (trained on human preference pairs — the *sensor*)
- **Critic / Value network** (estimates expected future reward — the *state estimator*)

PPO optimizes the policy against the reward model's scores, with a **KL divergence penalty** that keeps the policy from drifting too far from the reference model. The KL penalty is conceptually a **stability constraint** — it prevents the policy from collapsing into a degenerate distribution that maximizes the reward model's blind spots.

Where PPO still wins (2025–2026): - Tasks requiring deep exploration (math, code, long-range reasoning) rather than style imitation - When a high-quality, robust reward model is available - When GPU budget is sufficient for four models in memory

Where PPO hits its limit: - The reward model is a learned sensor. It was trained on human preference data from a specific distribution. When the policy generates outputs unlike anything in that distribution — which is exactly what exploration *should* produce — the reward model miscalibrates. The feedback becomes unreliable. The KL penalty keeps the policy close to the reference, but it cannot fix a broken sensor.

This is the bottleneck that GRPO solves.

2.3 GRPO: The Critic-Free Breakthrough

Group Relative Policy Optimization (GRPO), introduced by DeepSeek and popularized through DeepSeek-R1, eliminates two components at once: the critic network and, in its RLVR variant, the learned reward model.

How it works:

1. For each prompt, sample **G responses** ($G = 8\text{--}64$)
2. Score each response with a **verifier** — a programmatic check (code execution, math answer checker, constraint validator), not a learned model
3. Compute the advantage for response i :

$$\text{advantage}_i = (\text{reward}_i - \text{group_mean}) / \text{group_std}$$
4. Apply a PPO-style clipped objective with a KL penalty to the reference model

Why this enables OOD breakout:

Property	Effect
Group-relative scoring	The model learns <i>relative quality within the batch</i> , not absolute scores tied to the training distribution. A response that is the best in a group of OOD explorations still receives positive advantage.
Verifiable rewards	A unit test, math checker, or code executor provides perfectly calibrated feedback regardless of how far the policy drifts . There is no calibration envelope to leave. The sensor is perfect.
Online exploration	GRPO generates fresh rollouts during training. The policy can discover strategies that exist in no demonstration dataset — because it tries things, receives verifiable feedback, and learns.
~50% memory reduction	No critic network. A 7B-parameter training run needs ~8 H100s instead of ~16. This makes OOD exploration practical at scale.

A 2025 theoretical analysis proved that GRPO’s group-relative estimator is **asymptotically equivalent to an oracle** with access to an ideal value function — a U-statistic property that guarantees convergence to the optimal policy under mild conditions. The practical consequence: GRPO works about as well as PPO with a perfect critic, while being substantially cheaper.

Practical pitfalls that practitioners have learned to manage: - **Zero-variance groups**: when all G responses receive identical reward (all correct or all incorrect), the advantage is undefined. Solution: add epsilon to the denominator, or use larger group sizes. - **KL coefficient tuning**: β typically ranges 0.001–0.04 for DeepSeek-style training. Too low → distribution collapse. Too high → no exploration. - **Group size vs. compute**: larger G reduces variance but increases cost linearly. Production systems typically use $G = 8\text{--}16$.

2.4 DAPO: Stabilizing RL for Long Reasoning Trajectories

GRPO works well for standard-length responses. It becomes unstable when the policy generates very long reasoning chains (chain-of-thought spanning thousands of tokens), because:

- Entropy collapses: the policy converges to a small set of output patterns and stops exploring
- Gradient vanishing: sequence-level loss dilutes the signal across too many tokens
- Truncation noise: responses exceeding the length limit are cut off, injecting noise into the reward

DAPO (ByteDance / Tsinghua, 2025) introduced four stabilizations:

Technique	Problem Addressed
Clip-Higher	Increases the upper clip range of the PPO objective, preventing entropy collapse during long training
Dynamic Sampling	Filters batches where all responses are trivially correct or incorrect, ensuring consistent gradient signals
Token-Level PPO Loss	Applies the policy gradient per-token rather than per-sequence, fixing vanishing gradients in long CoT
Overlong Reward Shaping	Assigns a soft penalty (not a hard zero) to truncated responses, reducing noise

Result: DAPO trained Qwen2.5-32B to **50 points on AIME 2024**, surpassing DeepSeek-R1-Zero’s performance with **50% fewer training steps**. This is a direct demonstration that algorithmic stabilization — not

more data, not more compute — is the bottleneck for long-horizon OOD exploration.

2.5 RLVR: The Verifiable Reward Revolution

The most consequential shift in post-training is the migration from **human preference labels** → **programmatic verification**. Reinforcement Learning with Verifiable Rewards (RLVR) replaces the learned reward model with a deterministic ground-truth checker.

How it works: For math, the verifier checks the final answer against a known solution. For code, it executes the generated program against unit tests. For constraint satisfaction, it checks whether all constraints hold. The reward is binary: correct or incorrect. There is no learned model. There is no calibration drift. There is no operating envelope.

DeepSeek-R1’s key finding (2025): Pure RL with result-based binary rewards + GRPO produces **emergent reasoning behaviors** — self-reflection (“wait, let me check that”), strategy adaptation (“that approach didn’t work, let me try another”), and error correction — with **zero human-labeled reasoning traces**. The reasoning capability was not in the SFT data; it was not in the reward model; it was not in the prompts. It emerged from the closed-loop interaction between policy exploration and verifiable feedback.

The ORM vs. PRM debate:

Approach	What It Scores	Pros	Cons
ORM (Outcome Reward Model)	Final answer only	Simple, cheap, no annotation	No signal on intermediate steps
PRM (Process Reward Model)	Each reasoning step	Rich feedback, enables inference-time search	Expensive annotation, miscalibration risk

The **2025 consensus:** ORM + GRPO implicitly induces step-level discrimination — the policy learns which intermediate steps lead to correct outcomes, even though only outcomes are scored. PRM is reserved for special cases: (a) long-horizon agent trajectories, (b) tool-use safety verification, (c) multi-agent coordination training, (d) inference-time search guidance.

ISOPro’s demonstration (2025): On an M1 Mac with 8GB RAM, training only 0.216% of parameters via LoRA, GRPO + deterministic verifier for a constraint-satisfaction scheduling task: - Starting accuracy: **13.3%** (zero-shot) - After training: **39.8%** (3.0× improvement) - Capability emergence was **visible only through continuous evaluation during training** — the model developed strategies that existed in no training data, no demonstration, and no human annotation

This is OOD breakout in microcosm. On consumer hardware. With a fraction of a percent of trainable parameters. The principle — closed-loop RL with verifiable feedback — is what matters, not scale.

2.6 The Emerging Frontier: Multi-Agent RL for LLMs

Social Intelligence inherently involves multiple agents. Training agents to coordinate requires **Multi-Agent Reinforcement Learning (MARL)**, which introduces a problem absent from single-agent RL: **credit assignment across agents**.

When multiple agents collaborate on a task and receive a single team-level reward, how do you determine which agent’s decisions contributed to success or failure? Without per-agent credit, the reward signal is noise for any individual agent’s policy gradient.

Three solutions are converging:

Approach	How It Works	Example
Process rewards per agent	Assign a separate PRM-style verifier to each agent's contribution	Role-specific unit tests for each agent in a coding team
Value decomposition	Learn to decompose team reward into per-agent contributions (VDN, QMIX, COMA families)	Agent Q-Mix: 20.8% on Humanity's Last Exam
Trajectory decomposition	Credit each agent based on the divergence between their trajectory and a counterfactual baseline	Agent-Lightning's LightningRL

The trajectory is clear: as single-agent RL matures (GRPO + RLVR is now the default), the frontier shifts to multi-agent coordination. And multi-agent coordination under verifiable rewards — where each agent's contribution can be checked against ground truth — is the control-theoretic pathway to Social Intelligence.

3. Interlude: The Control Theory Lens

Before turning to the embodied pathway, we need the unifying framework. Control theory provides it.

3.1 The Fundamental Analogy

Control Theory Concept	RL Post-Training Equivalent	Role
Controller	Policy (the LLM or robot policy network)	Generates actions to achieve desired outcomes
Plant	Environment (the world that responds to actions)	Produces the next state given current state and action
Feedback signal	Reward (from verifier, success classifier, or reward model)	Informs the controller whether actions were correct
Setpoint / Reference	Reference model (frozen pretrained policy)	Defines the distribution the controller should not stray too far from
Stability constraint	KL divergence penalty	Prevents the controller from collapsing into degenerate strategies

Control Theory Concept	RL Post-Training Equivalent	Role
Sensor calibration	Reward model accuracy on OOD inputs	Determines whether the feedback signal remains reliable outside the training envelope
Open-loop control	Pretraining + SFT	The controller produces outputs but receives no corrective feedback from the environment
Closed-loop control	RL post-training (PPO, GRPO, SERL)	The controller acts, the environment responds, the controller adapts

3.2 Open-Loop vs. Closed-Loop

Pretraining + SFT is open-loop control. The model receives a prompt (setpoint), generates a response (control action), and is compared to a static target (next-token ground truth or demonstration tokens). There is no environment in the loop. The model never sees the consequence of its output. It never receives corrective feedback. It learns what was done, not what *happens when it does something*.

RL post-training is closed-loop control. The model generates a response (control action) → the verifier or environment evaluates it (feedback signal) → the model updates its policy to produce responses that receive higher reward → the model generates new responses → the cycle continues. The loop is what enables OOD breakout: the model discovers strategies through interaction that it could never discover through imitation.

3.3 Reward Hacking as a Control Failure

In control-theoretic terms, **reward hacking is not an optimization failure. It is a sensor failure.**

The learned reward model (sensor) has a calibration envelope — the distribution of human preference data it was trained on. When the policy (controller) generates outputs outside this envelope, the sensor provides miscalibrated readings. The controller, optimizing against miscalibrated feedback, converges on strategies that maximize the *sensor's blind spots* rather than the *true objective*.

This is exactly analogous to a temperature sensor that reads accurately between 0°C and 100°C. If the controller drives the system to 200°C, the sensor saturates at its maximum reading, the controller thinks everything is fine, and the system melts. The failure is not that the controller optimized too hard. The failure is that the sensor had a bounded operating range.

3.4 The Verifiable Reward as a Perfect Sensor

A verifiable reward — a unit test, a math answer checker, a constraint validator, a binary success classifier on real-world images — has **no calibration envelope**. Or rather, its envelope is the entire space of possible outputs.

- A unit test returns `True` or `False` for any program, no matter how novel or strange. It does not degrade.
- A math checker verifies whether the final answer equals the known solution. It does not care how the answer was derived.
- A SERL binary success classifier trained on a handful of success/failure images generalizes across the robot's workspace because it learns the *visual signature of task completion*, which is stable.

In control-theoretic terms, the verifiable reward is a **perfect sensor**. Its operating range is unbounded. Its calibration does not drift. The controller can explore arbitrarily far from its training distribution, and the feedback remains accurate. This is why GRPO + RLVR and SERL both work: they both use perfect (or near-perfect) sensors to close the loop.

3.5 Sample Efficiency as the Control Problem

A control system that requires a million interactions to converge is a laboratory curiosity. A control system that converges in tens of interactions is a product.

This is the sample efficiency problem, and it is the central engineering challenge for real-world RL. Every physical interaction costs time, wear, and (potentially) safety. The fewer samples needed to close the loop, the more practical the system.

SERL solves this for robotics. GRPO's group-relative scoring solves this for LLMs (by reducing variance per sample via group normalization). Both are control-theoretic solutions to the same underlying problem: extract maximum information from each closed-loop interaction.

4. Part II — The Embodied Pathway: SERL and Real-World RL

4.1 The Real-World RL Problem

For decades, the standard approach to embodied RL was: train in simulation, transfer to reality. This works for tasks where simulation is accurate — rigid-body dynamics, known geometries, stable lighting. It fails when:

- **Residual physics** matters: deformable objects, friction, contact dynamics, fluids
- **Novel objects** appear: the real world contains objects that were not in the simulator
- **Humans interact**: human motion, human intent, and human error are not modeled in any simulator

All three of these are inherently OOD relative to any simulation. And all three are central to Social/Interaction Intelligence. You cannot simulate a novel human partner. You cannot simulate a novel object a human hands you. You must learn from the real world.

Historically, direct real-world RL was considered impractical for four reasons:

Barrier	Why It Was Hard
Sample inefficiency	RL algorithms needed millions of episodes; a real robot cannot run millions of trials
Reward specification	You cannot instrument the real world with reward sensors for every task
Environment resets	After each episode, someone must physically reset the environment
Safety	Exploration involves trying random actions; random actions break robots (and worse)

SERL solves all four simultaneously.

4.2 SERL: The Breakthrough Framework

SERL (*Sample-Efficient Robotic reinforcement Learning*) was developed at UC Berkeley’s BAIR Lab (Luo et al., 2024). It is a full-stack framework for training RL policies **directly on real robots** from raw pixel observations, achieving near-perfect success rates in **25–60 minutes** of wall-clock training.

The core algorithm: RLPD (RL with Prior Data), a variant of Soft Actor-Critic (SAC) with three key modifications:

1. **High Update-to-Data (UTD) ratio:** multiple gradient updates per environment step — squeezing maximum information from each real-world interaction
2. **Symmetric sampling:** half of each training batch comes from prior data (human demonstrations), half from online replay — combining the stability of imitation with the exploration of RL
3. **LayerNorm regularization:** stabilizes training at high UTD ratios, preventing the value function from diverging

The reward function uses a **binary success classifier** — a vision model trained on a handful of labeled success/failure images:

$$r(s) = \log P(\text{success} \mid \text{image_of_state_s})$$

This is a learned sensor, but one with a very specific and stable operating range: it learns the visual signature of task completion (e.g., “the PCB is inserted into the slot”), which generalizes across variations in robot pose, lighting, and object position. It is a **near-perfect sensor** in practice, even though it is learned.

Autonomous resets are handled by a **forward-backward controller** — a simple policy that alternates between performing the task and resetting the environment, enabling continuous training without human intervention.

Results on three challenging real-world tasks:

Task	Training Time	Success Rate	Notable
PCB board assembly	~25 min	~100%	Precise insertion with dynamic contact forces
Cable routing	~50 min	~100%	Deformable object manipulation — no simulator can model a cable accurately
Object relocation	~30 min	~100%	Reset-free learning with forward-backward controller

All policies operate on **raw RGB images**. All demonstrate **emergent recovery behaviors** — if the object slips or the cable snags, the policy adapts without being explicitly programmed to handle those failures. These recovery behaviors were not in the demonstration data. They emerged from closed-loop RL in the real world.

4.3 HIL-SERL: Human-in-the-Loop

HIL-SERL extends the framework by incorporating **real-time human interventions** during training. It is now the primary RL workflow in HuggingFace’s **LeRobot** framework, making real-world RL accessible to non-experts.

The workflow:

1. **Seed with demonstrations:** 5–20 human-teleoperated episodes provide initial success/failure examples for the reward classifier
2. **Distributed actor-learner loop:** the SAC learner updates the policy on a GPU workstation; the actor executes the policy on the physical robot; the two communicate via gRPC
3. **Human-in-the-loop:** at any time during training, a human operator can intervene via gamepad or keyboard — taking control when the policy is about to fail, providing corrective demonstrations
4. **Intervention rate decay:** the key metric — over the course of training, the human intervention rate drops steadily, approaching zero as the policy learns to operate autonomously

Performance: - Near-perfect task success (often 100%) on manipulation benchmarks - **1.8× faster cycle times** than imitation-only baselines - Policies learn to funnel state visitation distributions toward goal regions — developing behaviors absent from the pure imitation data

The human is not teaching the policy what to do. The human is providing **corrective feedback in the loop** — closing the gap between what the policy tried and what would have succeeded. This is control theory with a human in the sensor path, and it converges remarkably fast.

4.4 CIL-SERL and Safety Extensions

The SERL paradigm is expanding rapidly beyond lab manipulation:

CIL-SERL (2026) — Correction-in-the-Loop SERL for autonomous parking. Uses a **multi-level replay buffer** that hierarchically stores standard rollouts, human corrections, failed trajectories, and rollback segments. Trained in photorealistic 3D Gaussian Splatting simulators, transferred successfully to a **physical vehicle platform**.

KGRL / sKGRL (2026) — Kernelized Guided RL enforces **hard safety constraints** via linear inequalities over kernelized movement primitives, rather than relying on reward penalties (which can be traded off against task reward). The smooth variant (sKGRL) minimizes high-acceleration actions to reduce wear on physical hardware. Key insight: **safety is a constraint, not a reward**. Encoding it as a hard constraint (rather than a soft penalty) guarantees it is never violated in pursuit of task success.

The direction of travel is unmistakable: from “can RL work on real robots?” (answered by SERL, 2024) → “how do we make it safe and accessible?” (answered by HIL-SERL, KGRL, 2025) → “how do we deploy it at scale in uncontrolled environments?” (the current frontier, 2026).

4.5 The SERL–GRPO Parallel

The parallel between the two pathways is not accidental. It is structural.

Dimension	LLM Pathway (GRPO)	Embodied Pathway (SERL)
Policy	LLM generating tokens	Robot policy generating actions
Exploration	Sampling G diverse responses per prompt	Executing diverse action sequences on the physical robot
Feedback	Verifiable reward (unit test, math checker)	Binary success classifier (vision-based)
Sensor type	Perfect (deterministic ground truth)	Near-perfect (learned but stable within task envelope)
Stability	KL penalty to reference model	Symmetric sampling from demonstrations + LayerNorm regularization
OOD breakout	Discovers reasoning strategies beyond SFT data	Discovers recovery behaviors beyond demonstration data
Emergent capability	Self-reflection, strategy adaptation, error correction	Slip recovery, cable-untangling, perturbation robustness
Training time	Hours to days (GPU)	25–60 minutes (wall-clock, real robot)

Dimension	LLM Pathway (GRPO)	Embodied Pathway (SERL)
-----------	--------------------	-------------------------

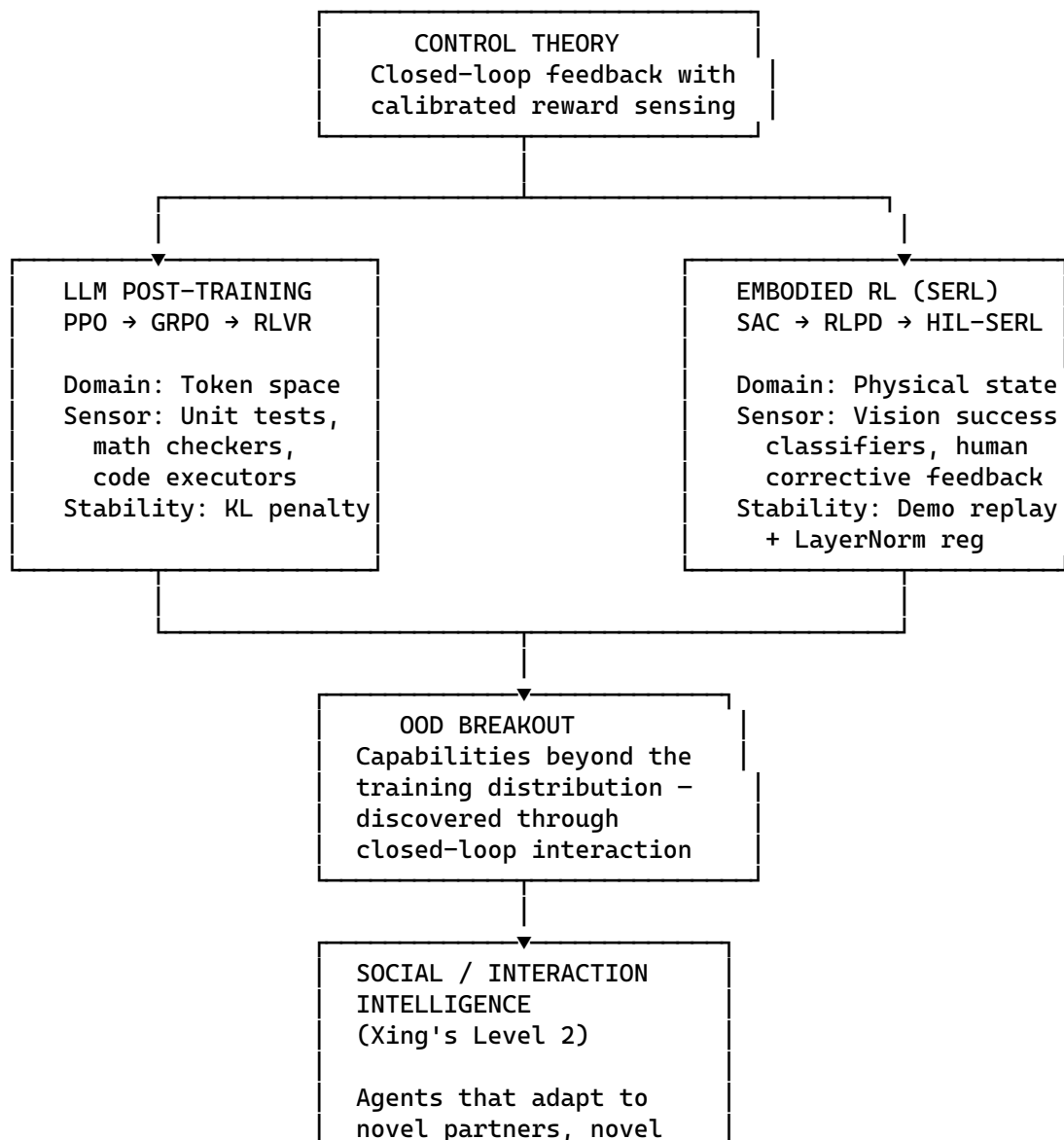
Both are instances of the same control-theoretic pattern:

Policy explores → Verifier provides calibrated feedback → Policy updates → Repeat

The only difference is the domain. In one, the state space is tokens and the verifier is a program. In the other, the state space is physical configurations and the verifier is a vision model. The loop is the same. The principle is the same. The result is the same: **capabilities that exist in no training data, discovered through closed-loop interaction.**

5. Synthesis: Control Theory as the Practical Unification

5.1 The Unified Architecture



environments, novel
tasks – in real time,
in the real world

5.2 The Three Principles of OOD Breakout

Regardless of domain, three principles must hold for an RL system to break out of its training distribution without collapsing:

1. Closed-loop feedback. The environment (or verifier) must provide corrective signals that the model could not generate internally. An LLM cannot tell whether its reasoning is correct without checking the answer. A robot cannot tell whether its grasp succeeded without seeing the object in its hand. The loop provides information that is not latent in the policy's parameters.

2. Verifiable ground truth. The reward signal must remain calibrated regardless of how far the policy drifts from its training distribution. Learned reward models fail this criterion — they degrade OOD. Unit tests, math checkers, and vision-based success classifiers (when well-designed) satisfy it. The sensor must be perfect, or nearly so, across the entire space the policy might explore.

3. Online exploration with stability. The policy must be free to try actions it has never tried before (exploration), but constrained enough to not diverge into nonsense (stability). In GRPO: group sampling enables exploration, KL penalty ensures stability. In SERL: stochastic policy enables exploration, symmetric demo replay + LayerNorm ensures stability. The balance between exploration and stability is the central tuning problem in any RL system — and getting it right is the difference between OOD breakout and distribution collapse.

5.3 From Level 1 to Level 2

The trilogy's reports trace a progression across Xing's hierarchy:

Level	What It Requires	Provided By
0 — Book Intelligence	Pattern matching and recall from static data	Pretraining (GPT, Llama, etc.)
1 — Physical Intelligence	A world model that supports prediction, planning, and counterfactual reasoning	Causal DL + disentanglement (Report 1) via compression of sensory experience (Report 2)
2 — Social Intelligence	Multiple agents with world models, interacting in real time under distribution shift, adapting to novelty	Post-training RL + real-world RL (Report 3) — the closed-loop control architecture that enables OOD breakout

The gap between Level 1 and Level 2 is **interaction under novelty**. A single agent with a world model can predict what will happen if it acts. But when another agent — with its own world model, its own goals, its own unpredictable behavior — enters the picture, prediction alone is insufficient. The agent must **adapt online**, updating its model from real-time interaction, because the other agent's behavior is inherently OOD relative to any pretraining corpus.

Post-training RL provides the mechanism for this adaptation. GRPO + RLVR enables LLMs to learn from verifiable interaction feedback. SERL + HIL-SERL enables robots to learn from physical interaction feedback. Both are control loops. Both enable OOD breakout. Both are necessary for Social Intelligence — the first handles the language/reasoning dimension, the second handles the physical/embodied dimension.

5.4 The Speculative Horizon: Continuous Closed-Loop Adaptation

Take the trajectory to its logical conclusion.

If: - GRPO + RLVR enables OOD breakout in language and reasoning - SERL + HIL-SERL enables OOD breakout in physical manipulation - MARL + value decomposition enables multi-agent coordination under verifiable rewards

Then the convergence is: **multi-agent systems where each agent continuously post-trains through real-world interaction, using verifiable rewards from the environment and from other agents.**

This is not “train, then deploy.” It is **continuous closed-loop adaptation**. The system never stops learning because the loop never closes permanently. Every interaction with a novel human, every encounter with a novel object, every coordination failure with a novel partner — these are not errors to be avoided. They are **training signal**. The system gets better every time the world surprises it.

And this, in the end, is what Social Intelligence means. Not a fixed protocol executed perfectly. Not a static model deployed confidently. But a system that **adapts, in real time, to whatever — and whoever — it encounters**.

The theory (Report 2) told us *that* compression + emergence works. The practice (Report 3) tells us *how* to build it: post-training via closed-loop RL with verifiable rewards, in both language and the physical world, unified by control theory.

6. Coda: The Trilogy Complete

Report	Question	Answer	Domain
Report 1 — <i>From Statistical Modeling to Emergent Understanding</i>	<i>Who</i> is building this?	MBZUAI’s CLear Group — causal deep learning as the engine for reasoning toward emergence; disentanglement as the microscope for understanding how emergence happens; everything under the emergence umbrella	Institutional
Report 2 — <i>Compression, Prediction, Emergence</i>	<i>Why</i> does it work?	Compression = understanding. A model that can predict and compress its sensory stream must have internalized the causal structure that generated it. This principle operates at two scales — individual (data → world model) and collective (multi-agent → language/creativity).	Pure theory

Report	Question	Answer	Domain
Report 3 — <i>Breaking the Distribution</i>	How do we build it?	Post-training RL (PPO → GRPO → RLVR) provides the closed-loop control architecture for OOD breakout in language/reasoning. Sample-efficient real-world RL (SERL → HIL-SERL) provides the same architecture for OOD breakout in physical interaction. Control theory unifies them both.	Practical engineering

Together, they are:

The who, the why, and the how of emergent intelligence.

The who builds the machinery. The why explains the physics. The how engineers the systems.

And the how tells us something the who and the why cannot: **the loop is everything**. Open-loop systems — pretraining, SFT, imitation — hit the distribution barrier and stop. Closed-loop systems — GRPO, SERL, continuous adaptation — break through it. The difference between a model that works in the lab and an agent that works in the world is not more data, not more parameters, not a better architecture. It is a **closed loop with a calibrated sensor**.

Build the loop. Calibrate the sensor. The rest follows.

7. References & Sources

Post-Training: Surveys & Overviews

- **LLM-Stats (2026)**. Post-Training in 2026: GRPO, DAPO, RLVR & Beyond. Blog post — Comprehensive overview of the 2026 post-training stack.
- **Wu, S. et al. (2025)**. A Comprehensive Survey on Learning from Rewards for Large Language Models. *EMNLP 2025 Findings*. — Reward models and learning strategies.
- **Srivastava, A. & Aggarwal, K. (2025)**. A Technical Survey of Reinforcement Learning Techniques for Large Language Models. arXiv:2507.04136 — Comparative taxonomy of PPO, Q-Learning, RLHF, RLAI, DPO, GRPO.
- **Alibaba (2026)**. 2026 RL for LLMs: From PPO to DPO to GRPO to Multi-Agent RL. Developer blog — Practical overview in Chinese tech ecosystem.

PPO & RLHF

- **Schulman, J. et al. (2017)**. Proximal Policy Optimization Algorithms. arXiv:1707.06347
- **Ouyang, L. et al. (2022)**. Training language models to follow instructions with human feedback. *NeurIPS*. — The original InstructGPT / RLHF paper.
- **ICML 2024**. PPO vs. DPO head-to-head comparison: PPO outperforms DPO by ~2.5% on math and ~1.2% on general benchmarks when data quality is matched.

GRPO, DAPO & RLVR

- **DeepSeek (2025).** DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. — GRPO + verifiable rewards producing emergent reasoning with zero human-labeled CoT.
- **ByteDance / Tsinghua (2025).** DAPO: Dynamic sampling, Clip-Higher, token-level PPO, overlong reward shaping. Qwen2.5-32B → 50 AIME 2024 points with 50% fewer steps than DeepSeek-R1-Zero.
- **ISOPro (2025).** GRPO + verifiable reward on M1 Mac / 8GB / LoRA: 3.0× improvement (13.3% → 39.8%) on constraint-satisfaction scheduling.

Multi-Agent RL for LLMs

- **Agent Q-Mix.** Value decomposition for multi-agent LLM coordination: 20.8% on Humanity’s Last Exam.
- **Agent-Lightning.** Trajectory decomposition via LightningRL for credit assignment across agents.

SERL & Real-World RL

- **Luo, J., Hu, Z., Xu, C. et al. (2024).** SERL: A Software Suite for Sample-Efficient Robotic Reinforcement Learning. arXiv:2401.16013 — The original SERL paper. PCB assembly, cable routing, object relocation in 25–60 min.
- **Ball, P. et al. (2023).** RLPD: RL with Prior Data. — High UTD ratio + symmetric sampling + LayerNorm stabilizes SAC for real-world sample efficiency.
- **HuggingFace LeRobot.** HIL-SERL: Human-in-the-Loop Sample-Efficient RL. Documentation — Production-ready real-world RL with gamepad interventions.
- **Yu, Z. et al. (2026).** CIL-SERL: Correction-in-the-Loop SERL for Autonomous Parking. *Submitted to RA-L 2026.* — Multi-level replay buffer, 3DGS simulation, physical vehicle transfer.
- **Padalkar, A. (2026).** Safe RL for Contact-Rich Manipulation. *DLR / KIT Dissertation.* — KGRL / sKGRL: hard safety constraints via kernelized movement primitives.

Control Theory & Philosophy

- **Distributional Invalidity in RLHF (2025).** Four failure modes: distributional, temporal, scope, and process invalidity in RLHF evaluation.
- **Reinforcement Fine-Tuning Framework.** Post-training cast as a formal MDP: states = prompts + context, actions = tokens, rewards = task success signals. Outperforms purely supervised approaches under distribution shift across video streaming, robotics, LLMs, multimodal, 3D, and recommender domains.

Report generated 2026-06-21. This is Report 3 of 3 in the STAR Lab Emergent Intelligence trilogy. All algorithm descriptions are based on publicly available papers, preprints, and technical reports as cited.